

SESSION 5

DOM Attributes, Styling & Array Methods

JS

JavaScript Notes

Session 5

DOM → Attributes → Events → Arrays → Methods

Notebook goal

Learn how JavaScript changes what users see in the browser, reads and writes HTML attributes, reacts to clicks, and works with arrays using common methods.

Reference basis: supplied Session 5 HTML, reference PDF, and visual flowchart. Wording and examples are rewritten in simple language.

Table of Contents

Clickable index - jump to a topic and use Return to Index on every note page

03 Session 5 Roadmap

04 DOM Element Selection

05 innerHTML

06 style Property

07 style.backgroundColor + fontSize

08 setAttribute()

09 getAttribute()

10 hasAttribute() + removeAttribute()

11 Image Attribute Switching

12 onclick + addEventListener()

13 Array Fundamentals

14 push()

15 pop()

16 shift()

17 unshift()

18 shift() vs unshift()

19 splice()

20 splice() - Add

21 splice() - Delete

22 splice() - Replace

23 join()

24 reverse()

25 sort()

26 Array Indexing + Second-Last

27 slice() + concat()

28 includes() / indexOf() / lastIndexOf()

29 forEach()

30 map()

31 filter()

32 find() + findIndex()

33 some() + every()

34 Modern Non-Mutating Array Options

35 Source Execution Walkthrough

36 DOM + Array Connection

37 Practice 1-10

38 Practice 11-20

39 Answer Key 1-10

40 Answer Key 11-20

41 Mini Project - Image Switcher

42 Mini Project - Array Manager

43 Interview Revision

44 Quick Revision Sheet

45 Session 5 Summary

46 Source Corrections & Clarifications

47 Clean Source HTML - Part 1

48 Clean Source HTML - Part 2

49 Clean Source HTML - Part 3

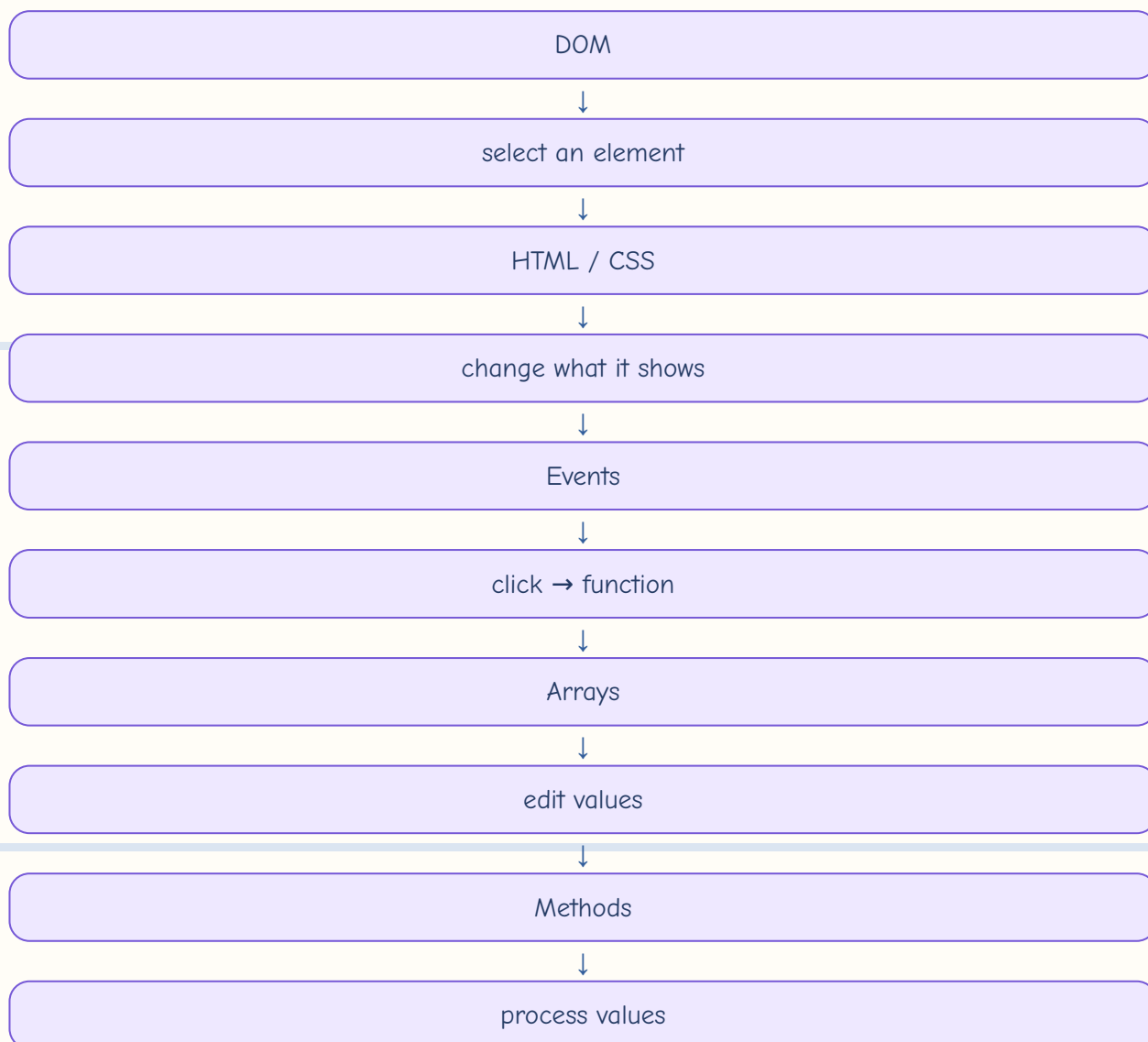
50 Reference Notes

Navigation tip

Click a topic. Every note page has a Return to Index button at the bottom.

Session 5 Roadmap

How the ideas connect



Core idea

The DOM part changes browser state. The array part changes or analyzes collections of values. In both cases, think: select → operate → inspect the result.

Visual roadmap

The supplied flowchart organizes the session into DOM interaction, array mutation, splice editing, conversion/reversal/access, and practice. These notes follow that learning path while using fresh examples.

DOM Element Selection

Start by selecting the element you want to change

Most browser updates begin with selecting an element. The supplied page gives the image id="img", so JavaScript can find it with `getElementById()`.

```
const image = document.getElementById("img");  
console.log(image);  
console.log(image.src);
```

Mental model

document → find element → receive an Element object → read or modify its properties.

Remember

Choose a selector that matches the HTML. An id is intended to identify one element.

innerHTML

Insert or replace HTML inside an element

```
const box = document.getElementById("main");  
box.innerHTML = "<div id='hey'>Hey</div>";
```

What happens

The supplied string is parsed as HTML, so a new child div is created inside #main.

Safer text

For plain text, `textContent` is usually better. Avoid inserting untrusted user input through `innerHTML` because it parses HTML.

style Property

Change CSS directly from JavaScript

```
const box = document.getElementById("main");  
box.style.color = "cyan";  
box.style.borderLeft = "5px solid cyan";  
box.style.borderRight = "5px solid purple";
```

CSS	JavaScript style name
background-color	backgroundColor
font-size	fontSize
border-left	borderLeft
margin-top	marginTop

Memory trick

CSS names usually become camelCase when used through element.style.

style.backgroundColor + fontSize

Update specific CSS properties

```
const title = document.getElementById("hey");  
title.style.backgroundColor = "purple";  
title.style.fontSize = "35px";
```

Read it

backgroundColor changes the background. fontSize changes the text size. Both are inline styles.

Better for larger UIs

When a design has reusable states, CSS classes and classList are usually easier to maintain than many inline styles.

setAttribute()

Create or change an HTML attribute

```
const image = document.getElementById("img");  
image.setAttribute("src", "new-image.jpg");  
image.setAttribute("alt", "A space image");
```

Simple rule

setAttribute(name, value) writes the attribute. If the attribute exists, its value changes; if it does not exist, the attribute is created.

Source connection

The supplied change() and reverse() functions use setAttribute("src", ...).

getAttribute()

Read an attribute value

```
const image = document.getElementById("img");  
console.log(image.getAttribute("src"));  
console.log(image.getAttribute("alt"));
```

Return value

You get the attribute value as a string. If that attribute is missing, the result is null.

Think

setAttribute = write. getAttribute = read.

hasAttribute() + removeAttribute()

Check or remove an attribute

```
const image = document.getElementById("img");  
console.log(image.hasAttribute("alt"));  
image.removeAttribute("alt");  
console.log(image.hasAttribute("alt"));
```

What happens

hasAttribute() returns true or false. removeAttribute() removes the named attribute.

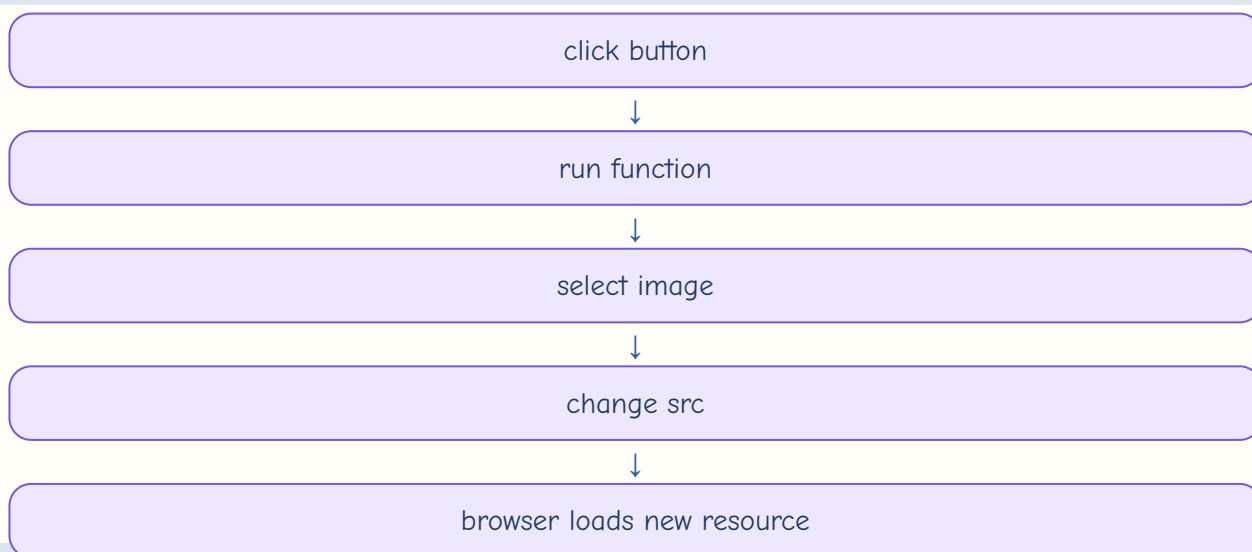
Useful pattern

Check first when your UI logic depends on whether an attribute exists.

Image Attribute Switching

Change an image when the user clicks a button

```
function changeImage() {  
  const image = document.getElementById("img");  
  image.style.transition = "1s linear";  
  image.setAttribute("src", "supernova.jpg");  
}  
  
function restoreImage() {  
  const image = document.getElementById("img");  
  image.setAttribute("src", "original.jpg");  
}
```



Important

Changing src swaps the image resource. A CSS transition does not automatically animate a src swap itself.

onclick + addEventListener()

Connect user clicks to JavaScript

```
<button id="btn" onclick="changeImage()">Change</button>

const btn = document.getElementById("btn");
btn.addEventListener("click", changeImage);
```

Source style

The supplied HTML uses inline onclick attributes. That is useful for learning the event flow.

Modern style

addEventListener("click", handler) keeps JavaScript behavior in JavaScript and allows multiple listeners.

Memory rule

onclick is the event-handler attribute name. With addEventListener(), use "click" without the on prefix.

Array Fundamentals

Arrays are ordered, mutable collections

```
const values = [11, 22, 33, 44, 55, 66];  
console.log(values[0]);  
console.log(values.length);
```

Group	Methods in this session
Add / remove ends	push, pop, shift, unshift
Edit by index	splice
Convert / rearrange	join, reverse, sort
Read / search	length, indexing, includes, indexOf, lastIndexOf
Process	forEach, map, filter, find, findIndex, some, every

Source correction

The supplied code writes `li = [...]` without `let` / `const` / `var`. Declare the variable explicitly.

push()

Add items at the end

```
const values = [11, 22, 33];  
const newLength = values.push(44, 55);  
console.log(newLength);  
console.log(values);
```

Result

newLength is 5 and values becomes [11,22,33,44,55].

Return value

push() returns the new array length, not the array. It mutates the original array.

pop()

Remove the last item

```
const values = [11, 22, 33, 44];  
const removed = values.pop();  
console.log(removed);  
console.log(values);
```

Result

removed is 44. values becomes [11,22,33].

Return value

pop() returns the removed last element.

shift()

Remove the first item

```
const values = [11, 22, 33, 44];  
const first = values.shift();  
console.log(first);  
console.log(values);
```

Result

first is 11. values becomes [22,33,44].

Source correction

The removed first value comes from shift(), not unshift().

unshift()

Add items at the beginning

```
const values = [22, 33, 44];  
const newLength = values.unshift(11);  
console.log(newLength);  
console.log(values);
```

Result

newLength is 4. values becomes [11,22,33,44].

Important

Calling unshift() with no arguments does not add or replace anything. It simply returns the current length.

shift() vs unshift()

The two front-of-array methods

Method	Action	Returns
shift()	Remove first element	Removed element
unshift(value)	Add at beginning	New array length

```
const values = [22, 33, 44];
const removed = values.shift();
values.unshift(11);
console.log(removed);
console.log(values);
```

Memory trick

shift = take away from the front. unshift = put onto the front.

splice()

The all-purpose array editor

```
array.splice(start, deleteCount, item1, item2);
```

Goal	Pattern
Insert	<code>splice(index, 0, value)</code>
Delete	<code>splice(index, count)</code>
Replace	<code>splice(index, count, replacement)</code>

Important

`splice()` mutates the original array and returns an array containing removed elements.

splice() - Add

Insert without deleting

```
const values = [22, 33, 44, 55];  
values.splice(1, 0, 888);  
console.log(values);
```

Result

[22, 888, 33, 44, 55]

How it works

Start at index 1. Delete zero items. Insert 888. Existing values move right.

splice() - Delete

Remove a chosen number of items

```
const values = [22, 888, 33, 44, 55];  
const removed = values.splice(1, 4);  
console.log(removed);  
console.log(values);
```

Result

removed = [888,33,44,55] and values = [22].

Think

start + count = which items are removed.

splice() - Replace

Delete and insert in one step

```
const values = [22, 66, 345];  
values.splice(1, 1, 999);  
console.log(values);
```

Result

[22, 999, 345]

Think

Start at index 1 → remove one item → put 999 in that position.

join()

Convert an array into one string

```
const values = [22, 345, 345];  
const text = values.join("_");  
console.log(text);
```

Result

22_345_345

Mutation?

No. join() returns a string and does not change the array.

reverse()

Reverse the order in place

```
const values = [22, 345, 345];  
const result = values.reverse();  
console.log(result);  
console.log(values);
```

Result

Both result and values now show the reversed order because reverse() mutates the original array.

Safer copy

If the original order matters, copy first: `const copy = [...values]`.

sort()

Sort arrays correctly

```
const nums = [10, 2, 30, 4];  
nums.sort();  
console.log(nums);  
  
nums.sort((a, b) => a - b);  
console.log(nums);
```

Why the comparator?

Default sort compares values as strings. For ascending numbers, return $a - b$.

Source clarification

The supplied file creates `sortArray` but never calls `sort()`. The important numeric rule is still worth learning.

Array Indexing + Second-Last

Use length arithmetic

```
const values = [345, 345, 22];  
console.log(values[values.length - 2]);
```

length = 3



last index = $3 - 1 = 2$



second-last = $3 - 2 = 1$



values[1] = 345

Memory rule

Last index = length - 1. Second-last index = length - 2.

slice() + concat()

Non-mutating array companions

Method	Purpose	Mutates?
slice(start,end)	Copy a portion of an array	No
concat(values)	Create a combined array	No

```
const values = [11,22,33,44];  
console.log(values.slice(1, 3));  
console.log(values.concat([55,66]));
```

Why learn these?

Use them when you want to read or combine values without changing the original array.

includes() / indexOf() / lastIndexOf()

Find values in an array

```
const values = [11, 22, 33, 22];  
console.log(values.includes(22));  
console.log(values.indexOf(22));  
console.log(values.lastIndexOf(22));
```

Method	Meaning
includes()	Does the value exist?
indexOf()	First matching index
lastIndexOf()	Last matching index

Output

true, 1, 3

forEach()

Do something for every element

```
const values = [10, 20, 30];
values.forEach((value, index) => {
  console.log(index, value);
});
```

Callback data

The callback commonly receives value, index and the full array.

Important

forEach() is for side effects. It does not build the transformed array for you. Use map() for transformation.

map()

Create a transformed array

```
const values = [10, 20, 30];  
const doubled = values.map(value => value * 2);  
console.log(doubled);
```

Result

[20, 40, 60]

Rule

One input element → one output element. map() returns a new array.

filter()

Keep only values that pass a test

```
const values = [10, 15, 20, 25];  
const even = values.filter(value => value % 2 === 0);  
console.log(even);
```

Result

[10, 20]

Rule

The callback should return true for items you want to keep.

find() + findIndex()

Find the first match

```
const values = [10, 15, 20, 25];  
console.log(values.find(v => v > 18));  
console.log(values.findIndex(v => v > 18));
```

Method	Returns
find(predicate)	First matching value or undefined
findIndex(predicate)	Index of first match or -1

Output

20 and 2

some() + every()

Ask Boolean questions about an array

```
const values = [2, 4, 6];  
console.log(values.some(v => v > 5));  
console.log(values.every(v => v % 2 === 0));
```

Method	Question
some()	Does at least one element pass?
every()	Do all elements pass?

Connection to earlier sessions

These return Boolean results and work naturally with comparison and logical operators.

Modern Non-Mutating Array Options

Useful when you want immutable-style code

```
const nums = [10, 2, 30, 4];  
const sorted = nums.toSorted((a, b) => a - b);  
const reversed = nums.toReversed();  
  
console.log(nums);  
console.log(sorted);  
console.log(reversed);
```

Simple idea

`toSorted()` and `toReversed()` return new arrays, so the original `nums` stays unchanged.

Why add this page?

The supplied session focuses on `sort()` and `reverse()`. Current JavaScript also provides non-mutating alternatives for the same jobs.

Source Execution Walkthrough

Track the supplied array step by step

Step	Operation	Array after the operation
Start	const li = [11,22,33,44,55,66]	[11,22,33,44,55,66]
1	push(345,333)	[11,22,33,44,55,66,345,333]
2	pop()	[11,22,33,44,55,66,345]
3	shift()	[22,33,44,55,66,345]
4	unshift() with no args	unchanged
5	unshift() with no args again	unchanged
6	splice(1,0,888)	[22,888,33,44,55,66,345]
7	splice(1,4)	[22,66,345]
8	splice(1,1,345)	[22,345,345]
9	join("_")	"22_345_345"
10	reverse()	[345,345,22]

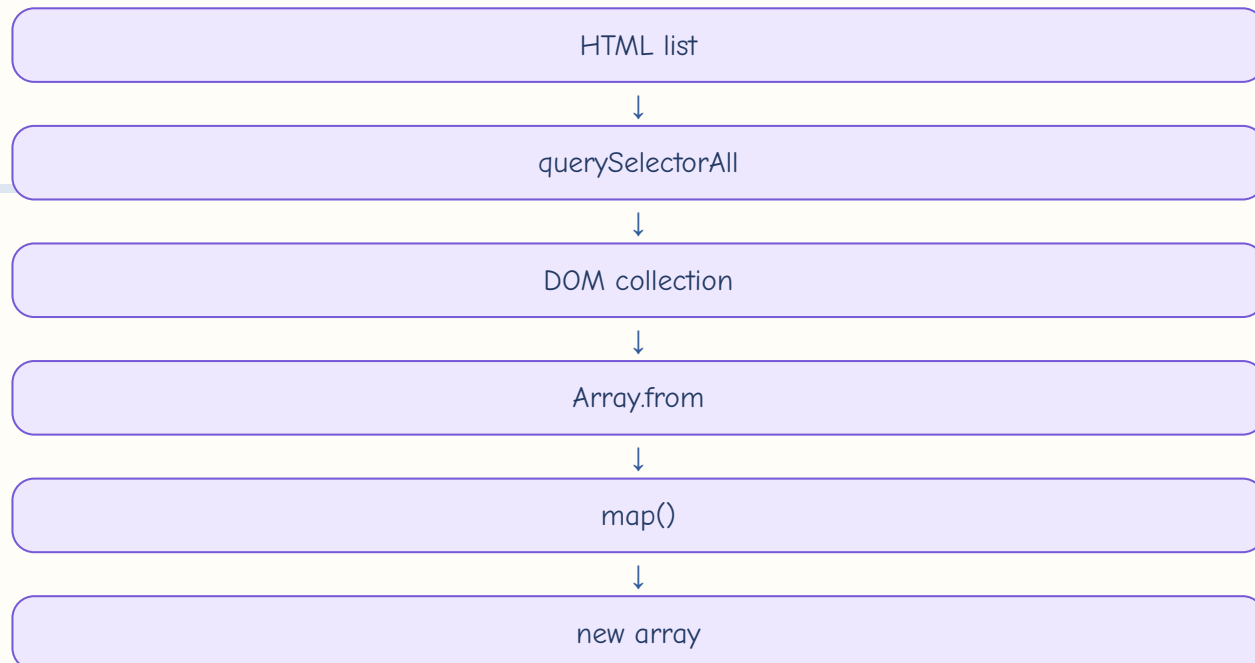
Key correction

The two unshift() calls do not add or replace anything because no values were supplied. The original source comments around those lines are misleading.

DOM + Array Connection

A practical browser pattern

```
const items = document.querySelectorAll("li");  
const names = Array.from(items).map(item => item.textContent);  
console.log(names);
```



Why it matters

DOM selection can produce collections. Converting them to arrays lets you use familiar array processing methods.

Practice 1-10

DOM attributes and style exercises

1. Select the image by id and print its src attribute.

2. Use `setAttribute()` to replace an image src.

3. Use `getAttribute()` to read alt text.

4. Check whether an image has a src attribute using `hasAttribute()`.

5. Remove the alt attribute with `removeAttribute()`.

6. Change an element background color using `style.backgroundColor`.

7. Change `fontSize` and `borderLeft` through style.

8. Use `innerHTML` to insert a small child div.

9. Explain the flow of `onclick="changeImage()"`.

10. Build two buttons: one changes the image and one restores it.

Practice 11-20

Array methods and indexing exercises

11. Add three values to the end of an array using `push()`.

12. Remove the last value and store it with `pop()`.

13. Remove the first value and display the removed item using `shift()`.

14. Add two values to the beginning using `unshift()`.

15. Insert a value at index 2 using `splice()`.

16. Delete two values beginning at index 1 using `splice()`.

17. Replace one element using `splice()`.

18. Convert an array to a string using `join("_")`.

19. Reverse an array and then access its second-last element.

20. Sort a numeric array in ascending order correctly.

Answer Key 1-10

Suggested solutions

```
1. const image = document.getElementById("img"); console.log(image.getAttribute("src"));
2. image.setAttribute("src", newUrl);
3. console.log(image.getAttribute("alt"));
4. console.log(image.hasAttribute("src"));
5. image.removeAttribute("alt");
6. element.style.backgroundColor = "purple";
7. element.style.fontSize = "35px"; element.style.borderLeft = "5px solid cyan";
8. element.innerHTML = "<div>Hey</div>";
9. Click → onclick → function → select image → set src.
10. Use two handlers and setAttribute("src", url) in each.
```

Practice tip

Expand each compact answer into a full working example while studying.

Answer Key 11-20

Suggested solutions

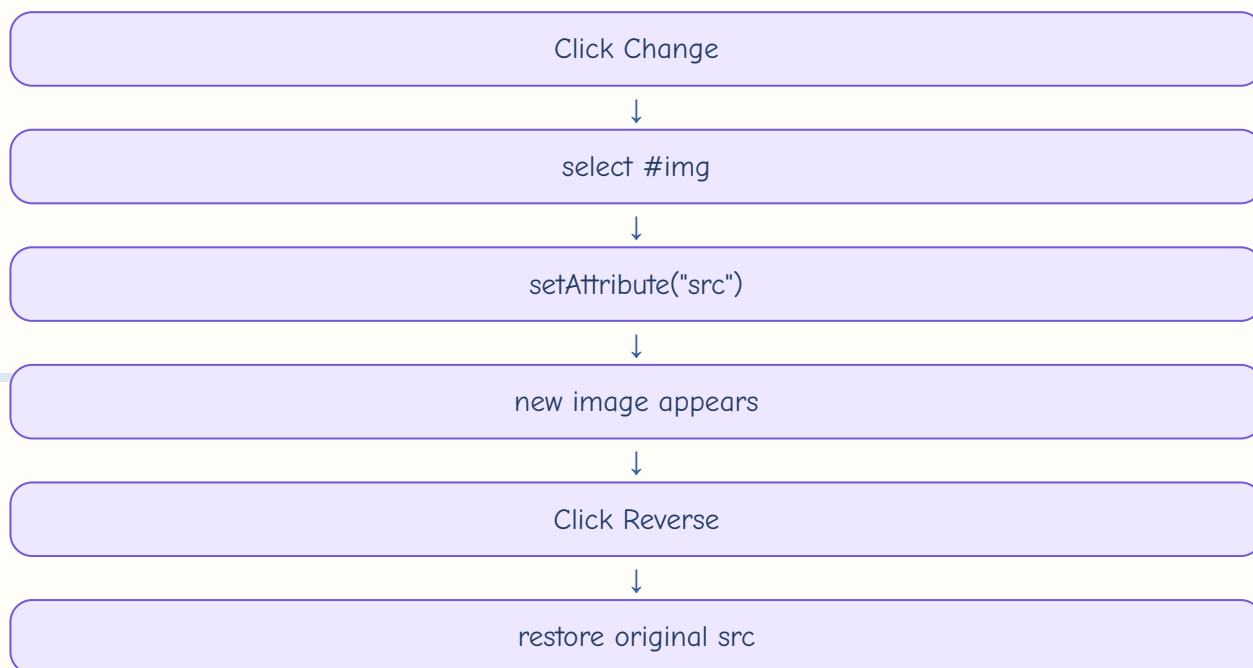
```
11. arr.push(10, 20, 30);
12. const removed = arr.pop();
13. const first = arr.shift();
14. arr.unshift(10, 20);
15. arr.splice(2, 0, value);
16. arr.splice(1, 2);
17. arr.splice(index, 1, replacement);
18. const result = arr.join(" ");
19. arr.reverse(); console.log(arr[arr.length - 2]);
20. arr.sort((a, b) => a - b);
```

Memory rule

push/pop work at the end. shift/unshift work at the beginning. splice handles index-based editing. join returns a string. reverse and sort mutate the original array.

Mini Project - Image Switcher

Combine selection, attributes and events



```
const image = document.getElementById("img");

document.getElementById("changeBtn").addEventListener("click", () => {
  image.setAttribute("src", "supernova.jpg");
});

document.getElementById("reverseBtn").addEventListener("click", () => {
  image.setAttribute("src", "original.jpg");
});
```

Target

Keep the image id unique and keep the original URL so the Reverse button can restore it.

Mini Project - Array Manager

Practice the complete array family

```
const values = [11, 22, 33, 44, 55];

// add end -> push()
// remove end -> pop()
// remove front -> shift()
// add front -> unshift()
// insert / delete / replace -> splice()
// array -> string -> join()
// reverse order -> reverse()
// numeric order -> sort((a,b) => a-b)
```

Extension

Put buttons on the page for each operation and display the array inside a <div> after every click. That combines arrays with the DOM.

Interview Revision

Fast answers for Session 5

Question	Answer
What does push() return?	The new array length.
What does pop() return?	The removed last element.
shift() vs unshift()?	Remove first vs add at beginning.
What does splice() do?	Insert, delete or replace values at an index.
Does join() mutate?	No. It returns a string.
Does reverse() mutate?	Yes. It reverses the original array.
Why use a comparator with sort()?	Default sorting is lexicographic; a-b gives numeric ascending order.
setAttribute()?	Sets or replaces an attribute.
getAttribute()?	Reads an attribute value.
innerHTML?	Parses HTML and replaces an element's contents.
onclick?	Runs a handler when the element is clicked.
forEach() vs map()?	forEach performs work; map creates a new transformed array.
filter()?	Creates a new array containing items that pass a test.
find() vs findIndex()?	First matching value vs first matching index.

Quick Revision Sheet

Session 5 at a glance

DOM

getElementById -> select one element by id
innerHTML -> insert / replace HTML
style.* -> set inline CSS properties
setAttribute -> set / change attribute
getAttribute -> read attribute
hasAttribute -> test attribute existence
removeAttribute -> remove attribute
onclick -> run handler after click
addEventListener -> modern event wiring

ARRAYS

push -> add end
pop -> remove end
shift -> remove front
unshift -> add front
splice -> insert / delete / replace
join -> array to string
reverse -> reverse in place
sort -> sort in place
slice / concat -> new arrays
forEach -> process each item
map -> transform
filter -> keep matches
find / findIndex -> first match
some / every -> Boolean tests

Memory map

DOM → select → read/change. Arrays → add/remove/edit → rearrange → process.

Session 5 Summary

What you should be able to do now

Skill	Expected ability
DOM selection	Select elements and work with element objects.
HTML content	Insert or replace markup with innerHTML.
Inline styling	Set CSS properties through element.style.
Attributes	Read, set, check and remove attributes.
Events	Connect clicks to JavaScript functions.
Array mutation	Use push, pop, shift, unshift and splice.
Array conversion	Use join to create a string.
Array rearrangement	Reverse and numerically sort values.
Array access	Use length and index arithmetic.
Array processing	Use forEach, map, filter, find and condition tests.

Next step

Combine these tools with loops, reusable functions, event listeners and DOM rendering to build interactive browser mini-projects.

Source Corrections & Clarifications

Important notes from the supplied code

Source item	Clarification
li = [...]	Declare the variable with const or let.
li.unshift() with no arguments	It does not change the content; it returns the current length.
"Get the first removed value using unshift()"	The removed first value comes from shift(), not unshift().
"To replace the first" + unshift()	unshift adds at the beginning. It does not replace an existing item.
sortArray created but not sorted	A sort call was missing; numeric sorting needs (a,b) => a-b.
reverse()	Mutates the original array.
setAttribute(src)	Changes the image resource when the URL is valid.
onclick	Works for learning; addEventListener is cleaner for larger applications.

Why this page exists

The supplied material is useful practice, but a few comments do not match the actual behavior of the method. These notes keep the original intent while making the examples technically usable.

Clean Source HTML - Part 1

Normalized version of the supplied HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Session 5</title>
</head>
<body>
  <button id="btn">Click to change</button>
  
  <button id="reverse">Reverse</button>
</body>
</html>
```

Study note

The source HTML and the reference flowchart both center the session around one image plus two buttons. This cleaned version removes repeated or unused code so the learning flow is easier to see.

Clean Source HTML - Part 2

The DOM interaction pattern

```
const image = document.getElementById("img");
const changeBtn = document.getElementById("btn");
const reverseBtn = document.getElementById("reverse");

changeBtn.addEventListener("click", () => {
  image.setAttribute("src", "supernova.jpg");
});

reverseBtn.addEventListener("click", () => {
  image.setAttribute("src", "original.jpg");
});
```

Flow

Click → event listener → select image → set src → browser updates the image.

Clean Source HTML - Part 3

The array method pattern

```
const values = [11, 22, 33, 44, 55, 66];

values.push(77);
const last = values.pop();
const first = values.shift();
values.unshift(10);
values.splice(1, 0, 88);
const text = values.join("_");
values.reverse();
values.sort((a, b) => a - b);

console.log(values, last, first, text);
```

Study note

For learning, use a fresh array for each practice problem. Chaining many mutating methods on one array is good for tracing, but not ideal for beginners.

Reference Notes

Where the clarifications came from

Provided files

Session 5 HTML and Session 5 PDF supplied in this chat, plus the supplied visual flowchart. The reference PDF contains the original topic structure, practice questions, answer keys and mini-project ideas.

MDN checks

Used for DOM attributes, event listeners and array method behavior.

W3Schools checks

Used as a second tutorial/reference source for DOM events and array method explanations.

MDN: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

W3Schools: <https://www.w3schools.com/js/>